

DSA4212 Project Report

Best Performance Within a Fixed Compute Budget: Model Size and Dropout in Piano Music-Event Transformers

MAESTRO v3.0.0 | Random initialization | NVIDIA A10

Name: Teo Chloe

Student ID: A0286502E

Email: e1249216@u.nus.edu

Course: DSA4212

Date: 8 November 2026

Project repository

https://github.com/chloeheh/DSA4212-project

1 Dataset and music-event tokenization

MAESTRO pairs expressive piano MIDI with aligned audio; the released data are organized by performance and accompanied by composer, title, year, and split metadata [1]. This project uses the *MIDI-only v3.0.0* data, not audio waveforms. Its 1,276 performances provide note timing, pitch, velocity, and pedal information without requiring an audio encoder. The supplied split assignments are preserved; the dataset documentation describes composition-disjoint subsets [2]. This reduces overlap of the same composition across splits, although it is not a new leakage audit performed by this project.

Table 1: Dataset sizes after this project's tokenizer; counts are read from data/maestro/summary.json.

Split	Pieces	Event tokens
Training	962	33,293,600
Validation	137	3,790,187
Test	177	4,334,269

Representing music as a discrete sequence

The notebook tokenization.ipynb converts each MIDI performance into integer event IDs. This makes autoregressive music modeling compatible with the categorical next-token objective used in language modeling. A token is an instruction, not necessarily a note: one note involves velocity, note-on, elapsed-time, and note-off events. The vocabulary contains 376 tokens:

Family	Count	Meaning
NOTE_ON	128	Start MIDI pitch 0–127.
NOTE_OFF	128	End MIDI pitch 0–127.
TIME_SHIFT	100	Advance 1–100 ticks; each tick is 10 ms.
VELOCITY	16	Set the velocity bin for subsequent note-ons.
PEDAL_ON/OFF	2	Change sustain state, from controller CC64.
BOS/EOS	2	Mark the beginning/end of a piece.

Long pauses are decomposed into shifts of at most one second. Notes are quantized in absolute elapsed seconds; simultaneous events are ordered as note-offs, pedal changes, then note-ons. The encoder emits a velocity token before every note-on. Pedal messages are emitted only when the sustain state changes. Each piece starts with BOS and ends with EOS.

Storage, checks, and information loss

The notebook writes train.bin, val.bin, and test.bin as uint16 IDs. meta.pkl stores the vocabulary and mappings; piece_offsets.json stores each piece's starting offset and length. Assertions verify file lengths, token ranges, and boundary markers. An example is decoded into roundtrip_example.mid for inspection; it is not a generated-model sample.

The representation is deliberately lossy: timing is rounded to 10 ms, velocities are binned, and original tempo/program metadata and controllers other than CC64 are omitted. Consequently, results concern prediction of this event representation, not faithful waveform reconstruction. Event counts are not durations: a 1,024-token context spans different amounts of musical time depending on event density.

2 Research question, models, and evaluation criteria

Objective. Train a small autoregressive model from randomly initialized weights and investigate how to obtain the *best performance within a fixed compute budget per training run.* Musical event sequences provide a structured discrete domain in which the language-model objective remains applicable. The study is not a pretrained-model fine-tuning exercise.

Hypotheses. First, increased parameter count could improve prediction through greater representational capacity, but may lose its advantage when fewer updates fit into the same time budget. Second, if larger models fit training data increasingly well while validation loss rises, stronger dropout may improve their generalization. Neither hypothesis assumes that the largest model must win.

Table 2: Dense decoder-only GPT variants. Counts include learned position embeddings and count tied token/output weights once.

Model	Layers	Heads	Width	Total parameters
Small	6	4	256	5,080,320
Medium	8	8	512	25,891,328
Large	12	12	768	86,029,056

Each run uses one NVIDIA A10 on Modal, with 24 GB device memory and BF16-capable Tensor Cores [3]. Models run sequentially, rather than using a multi-GPU cluster. All have vocabulary size 376, context length 1,024, and an effective batch of 32 sequences. These are relative size labels for this study, not standard pretrained GPT-2 variants.

Primary metric and checkpoint selection

For target events y_i and the context c_i available to the causal model, token-averaged negative log likelihood is

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(y_i \mid c_i), \qquad \text{PPL} = \exp\{\mathcal{L}(\theta)\}. \tag{1}$$

Natural logarithms give cross-entropy in nats per token; lower loss and perplexity are better. The metric rewards assigning probability to the actual next event, rather than only counting whether the top prediction is correct. It does not give partial credit for a musically nearby wrong pitch or establish perceptual quality.

During training, each validation estimate averages 20 batches of four 1,024-token windows: 81,920 target positions. Seeds reset at each evaluation so models face the same windows. Windows can overlap, so these positions are not necessarily distinct. Dropout and gradient computation are disabled for evaluation.

The primary checkpoint is best.pt, the lowest *recorded validation* loss within the budget; ckpt.pt stores the latest state. Selection never uses test loss. The best checkpoint may precede two hours, while the run still receives the full opportunity to improve. This directly matches "best within budget," rather than requiring the final weights to be optimal. Final losses remain useful diagnostics of overfitting. Full-coverage validation/test evaluation uses a separately specified, matched protocol in Section 9.

3 Adapting nanoGPT to MAESTRO

Why nanoGPT is an appropriate base

The implementation starts from nanoGPT's compact PyTorch GPT code [4]. Its explicit model definition and training loop make architecture, data handling, optimizer behavior, and checkpointing inspectable. This is useful for a controlled educational experiment: changing width, depth, and dropout does not require replacing

the whole framework. The choice is one of transparency and reuse, not evidence that nanoGPT is the fastest possible music-model implementation.

The retained decoder uses learned token and absolute position embeddings, pre-normalized causal self-attention, residual connections, and a GELU MLP expanding width by a factor of four. Token embeddings and the output head share weights. Causal masking prevents positions from reading their next-token targets; self-attention supplies the sequence modeling mechanism [5]. Models are initialized randomly; inherited text-GPT loading methods are not called.

Changes needed for this dataset

MaestroData memory-maps the token binaries and reads piece boundaries. It samples uniformly across admissible within-piece window starts. A piece of length L contributes $L-1{,}024$ starts when $L > 1{,}024$; longer pieces therefore receive proportionally more sampling opportunities. Inputs contain tokens $t_0, \ldots, t_{1023}$ and targets contain $t_1, \ldots, t_{1024}$. Neither crosses a piece boundary. Shorter pieces cannot supply full training windows.

The loader validates offsets, binary lengths, and vocabulary ranges, then gathers each 1,025-token window once to form contiguous input/target arrays. Training reads the true vocabulary size from metadata: GPTC onfig's inherited 50,304-token default is explicitly overridden by 376. Separate configuration files specify the three architectures. configurator.py applies these settings and command-line overrides before model construction.

Table 3: Shared training settings; the baseline uses dropout 0.1.

Optimizer	AdamW	Weight decay	0.1
(β_1, β_2)	$(0.9, 0.95)$	Gradient clipping	Norm 1.0
Peak / minimum LR	3×10^{-4} / 3×10^{-5}	Seed	1337
Precision	BF16 autocast	Effective batch	32 sequences
Context	1,024 tokens	Targets/update	32,768

AdamW uses adaptive gradient moments with decoupled weight decay [6]; matrix weights receive decay, while one-dimensional normalization parameters do not. Gradients are cleared with set_to_none=True; fused AdamW is enabled on CUDA. PyTorch scaled dot-product attention is used when available. torch.compile remains disabled; compiler modes were not benchmarked.

Budget enforcement and reproducibility

Each run receives 7,200 seconds of training-loop time, including initial/periodic validation and checkpoint writes. Initialization and final evaluation/save add overhead, and the current optimizer step is completed at the boundary. Thus this is an approximately two-hour loop budget, not an exact two-hour invoice or fixed-FLOP allocation. Atomic checkpoint writes retain weights, optimizer/scaler states, random states, elapsed budget, and token counts; resuming preserves the original budget. Modal Volumes persist outputs independently of the local client. The image pins Python 3.11, PyTorch 2.6.0, and NumPy 2.2.3.

4 Optimizing throughput under the time budget

Batch-size benchmarking without changing effective batch

A fixed hardware budget makes implementation efficiency part of the experiment. Before baseline training, microbatches of 4, 8, 16, and 32 were benchmarked on the A10. Accumulation steps were respectively 8, 4, 2, and 1, preserving 32 sequences and 32,768 targets per optimizer update. Each trial included the real loader,

transfer, forward/backward passes, clipping, and AdamW. Three warm-up steps preceded six measured steps; throughput used their median duration.

The selection rule maximized observed throughput below 85% of device memory. Microbatch 32 won for all architectures. BF16, fused optimization, and efficient batch gathering were retained. This is a measured batching improvement, not a global optimization of every kernel, model, or hyperparameter. The short benchmark and complete training loop also differ in evaluation/checkpoint overhead.

Table 4: A10 batching measurements. Throughput is target tokens/s; memory is peak allocated GB, not total reserved/device usage.

Model	Batch 4	Batch 32	Speedup	Batch-32 GB
Small	424,395	603,065	1.42×	2.24
Medium	151,006	182,301	1.21×	5.85
Large	57,946	68,590	1.18×	13.22

What the equal-time constraint actually equalizes

The same architecture-independent learning-rate schedule is indexed by elapsed fraction $u = t/7{,}200$. With warm-up fraction $w = 0.02$, it increases toward $\eta_{\max}$ during approximately 144 seconds, then follows

$$\eta(u) = \eta_{\min} + \frac{\eta_{\max} - \eta_{\min}}{2}\left[1 + \cos\left(\pi \frac{u - w}{1 - w}\right)\right], \quad u \geq w. \tag{2}$$

The code applies a small positive warm-up floor. Consequently, models have comparable learning rates at the same time, but not at the same update count. Validation and checkpoint time also advance this clock.

Table 5: Actual baseline training exposure within approximately 7,200 seconds. Equivalent passes are tokens seen divided by 33,293,600; sampling is with replacement, not ordered epochs.

Model	Updates	Targets seen (billions)	Equivalent passes
Small	127,396	4.175	125.4
Medium	40,719	1.334	40.1
Large	15,430	0.506	15.2

Small processes about 8.3 times as many targets as large. Increasing capacity therefore trades off against repeated data exposure and the number of optimizer updates. Equal A10 time is appropriate for a rental-time question, but does not isolate parameter count, equalize arithmetic FLOPs, or establish sample efficiency. A stronger architecture-only comparison would add an equal-token study; it would answer a different question and cost additional compute. The shared LR and dropout settings also improve control without proving that each architecture has its optimal settings.

5 Baseline learning curves: dropout 0.1

Appendix Figure A1 shows the 13 saved evaluations per architecture: initialization, approximately ten-minute intervals, and completion. Training losses here are also measured with dropout disabled on fixed sampled training windows; they are not noisy losses from a single optimization microbatch. The lower row omits initialization and uses a common scale to expose later changes.

Table 6: Baseline loss and best recorded time. All values are sampled cross-entropy in nats/token.

Model	Best val.	Best time (min)	Final train	Final val.
Small	**1.7652**	110.1	1.5603	1.7659
Medium	1.8018	40.1	1.3299	1.8892
Large	1.8122	50.3	1.2865	1.9224

All models improve rapidly before the first ten-minute measurement. Small continues improving gradually and largely plateaus late in the run. Medium and large reach their lowest validation losses much earlier; their training losses continue decreasing while validation deteriorates. Large obtains the lowest final training loss but the worst final validation loss. This separates fitting the observed training distribution from predictive performance on held-out pieces.

Under the stated checkpoint rule, the baseline ranking is small, medium, then large. The gap is modest: medium and large best-checkpoint perplexities are about 3.7% and 4.8% higher than small's. The evidence does not imply that additional parameters are intrinsically harmful; it shows that additional capacity did not win under this implementation, dataset, schedule, and time constraint.

6 Interpreting capacity, exposure, and generalization

The baseline validation overlay is in Appendix Figure A2.

Evidence for overfitting, not numerical divergence

The final validation-minus-training gaps are 0.2055, 0.5593, and 0.6360 for small, medium, and large. For the latter two, validation rises repeatedly after its minimum while training loss falls. This is consistent with overfitting, rather than loss explosion or numerical failure: the recorded losses remain finite and training continues to improve.

The gap is descriptive, not an independent estimate of generalization error: it compares fixed samples from two different splits. Its widening is more informative than its exact absolute size. No repeated-seed experiment was performed, so differences should not be labeled statistically significant.

Why the largest model need not win

Medium and large have approximately 5.1 and 16.9 times small's parameter count, while receiving fewer updates and fewer repeated training targets. Large still fits the training sample strongly, suggesting that lack of representational capacity is not the immediate bottleneck. However, lower training loss is insufficient to disentangle data exposure, implicit regularization, architecture, and optimizer effects. A shared learning rate is controlled but not necessarily equally suitable for all widths/depths.

These findings favor small as the observed compute-efficient baseline. They do not demonstrate convergence of every configuration, universal superiority of small models, or a scaling law. The outcome is bounded to roughly two A10 hours per run and this music-event vocabulary.

Implication for the next intervention

Checkpoint selection already avoids using deteriorated final weights: "best within budget" rewards an earlier successful state without pretending that all models peak simultaneously. It does not improve the best loss already reached. The next intervention therefore targets *lowering the validation minimum*, not merely postponing termination. Increasing dropout provides a controlled test of whether larger models can use their capacity with less reliance on training-specific feature combinations. Validation remains the adaptation signal; the test split is not consulted during this decision.

7 Follow-up experiment: stronger dropout

Rationale and mechanism

The baseline already includes dropout 0.1 and AdamW weight decay 0.1. The follow-up therefore tests *stronger existing regularization*, not regularization versus none. Dropout disrupts co-adaptation by masking activations during training [7]. For drop probability p, an inverted-dropout operation is

$$\widetilde{h} = \frac{m \odot h}{1-p}, \qquad m_j \sim \text{Bernoulli}(1-p). \tag{3}$$

Retained values are rescaled during training; dropout is disabled during validation and test inference. In this implementation, the shared probability controls embedding, attention-probability, attention-output, and MLP-output dropout. Changing it therefore modifies several existing sites together, not just one hidden layer.

Weight decay was a credible alternative: it shrinks parameters through AdamW's decoupled update [6]. But recorded losses do not show that insufficient weight shrinkage is the cause, and no weight-norm diagnostic was collected. Increasing dropout instead tests a complementary source of regularization while preserving optimizer settings. The choice is pragmatic under limited funding, not evidence that dropout must beat stronger weight decay. The value 0.2 is a moderate intervention, not a literature-established MAESTRO optimum.

Design: isolate one setting across all sizes

All three architectures are retrained from random initialization with dropout increased from 0.1 to 0.2. Small is included even though its baseline plateau is less concerning: it is a useful check on whether an intervention benefits every capacity equally. This adds three two-hour runs, rather than assuming medium's behavior transfers to the others.

Table 7: Controls for the paired dropout comparison within each architecture.

Factor	Treatment or control
Dropout	Only training hyperparameter changed: 0.1 → 0.2.
Architecture / initialization	Same layers, heads, width, vocabulary, and seed 1337 per size.
Optimizer / schedule	Same AdamW, decay 0.1, LR limits, clipping, and time schedule.
Batching / context	Reuse batch 32, accumulation 1, and context 1,024.
Evaluation / budget	Same fixed windows, frequency, checkpoint rule, and 7,200-second limit.

Frozen configurations and a dedicated launcher are stored in modified-nanoGPT/experiments/regularization/. The launcher checks each candidate against its saved baseline and rejects accidental changes beyond dropout and output destination. The prior batching result is reused, avoiding another paid sweep. Outputs remain separate from the baselines; interrupted jobs resume their original budgets.

Predefined improvement criterion

The primary question is whether dropout 0.2 achieves a lower *best validation cross-entropy* than dropout 0.1 for the same size. A smaller training–validation gap alone is not sufficient: stronger regularization can worsen both losses through underfitting. Final losses, time of the minimum, and processed tokens provide secondary diagnostics. The same seed controls initialization but does not remove stochasticity or guarantee identical masks and trajectories. This is a one-seed ablation, not a broad hyperparameter search.

8 Regularization findings and their interpretation

Appendix Figure A4 overlays each architecture's two dropout trajectories; full follow-up curves appear in Figure A3.

Table 8: Effect of dropout on sampled validation losses. Δ best is dropout 0.2 minus 0.1; negative favors the intervention.

	Best validation			Final validation	
Model	$p = 0.1$	$p = 0.2$	Δ best	$p = 0.1$	$p = 0.2$
Small	**1.7652**	1.7665	$+0.0013$	1.7659	1.7665
Medium	1.8018	**1.7872**	-0.0146	1.8892	1.8051
Large	1.8122	**1.7968**	-0.0154	1.9224	1.8305

A capacity-dependent benefit

Medium and large improve their best validation perplexities by approximately 1.45% and 1.53%. Their recorded minima shift from 40.1 to 60.1 minutes and from 50.3 to 60.4 minutes, respectively. Their final loss improvements, 0.0840 and 0.0919 nats/token, are larger than the improvements in minima. Thus dropout chiefly suppresses late deterioration while also giving a modest improvement to the primary selection metric.

The final training–validation gaps shrink from 0.5593 to 0.3841 for medium and from 0.6360 to 0.4572 for large. The gaps remain positive and validation still rises after the minima: overfitting is reduced, not eliminated. Because evaluation occurs about every ten minutes, the true optimal stopping times are not precisely identified.

Small's best loss changes by only $+0.0013$, corresponding to about 0.13% higher perplexity. Its final gap shrinks from 0.2055 to 0.1444, yet validation does not improve. This is a concrete counterexample to treating gap reduction as success. Stronger dropout initially slows all models, but appears better matched to the larger models' generalization problem. This is an interpretation of the curves, not a proven causal account of internal features.

Fairness and the scope of the finding

Follow-up token counts are 4.139B, 1.329B, and 0.507B, close to their baseline counterparts. Gross throughput changes therefore do not explain away the observed pattern, although equal time never guarantees equal training exposure. Small remains the lowest-loss architecture among both dropout settings on sampled validation. The evidence supports a size-dependent regularization effect in these runs, not that dropout 0.2 is universally optimal or superior to weight-decay tuning. Across both experiments, six runs consume approximately 12 training-loop GPU-hours, excluding benchmarking, startup, and final evaluation.

9 Final held-out evaluation and diagnostic evidence

The primary comparison uses the best validation-selected configuration among those tested for each size: **small dropout 0.1, medium dropout 0.2, and large dropout 0.2**, each using best.pt. Their checkpoint times are approximately 110.1, 60.1, and 60.4 minutes. This implements "best within the budget," not a requirement to use the final two-hour weights. Because dropout differs, this evaluates each size with its selected regularization, rather than isolating architecture alone.

Evaluation sequence. Initially all three dropout-0.2 checkpoints were tested. After inspecting those results, the small entry was replaced with dropout 0.1, which had already achieved lower validation loss before any test access. Only small inference was rerun; medium/large results were reused. The original small result and protocol are retained in an audit history, not treated as another training experiment. This revision is

disclosed rather than described as an entirely untouched pre-test protocol. No new hyperparameters were chosen using the revised test result.

Matched full-coverage scoring

Inference disables dropout and gradients. Every target after the first token of each piece is scored once, including EOS and partial tails, using consecutive blocks of at most 1,024 targets. Context resets at block boundaries and never crosses pieces. Padding is excluded and loss is token-weighted. This scores 3,790,050 validation and 4,334,092 test targets from 137 and 177 pieces. Validation is recomputed under the test procedure, rather than compared directly with the different random-window selection estimator.

Table 9: Revised primary comparison: validation-selected dropout and best checkpoint for each size.

Model	Dropout	Full validation loss	Test loss	Test perplexity
Small	0.1	**1.7581**	**1.6935**	**5.4385**
Medium	0.2	1.7848	1.7179	5.5730
Large	0.2	1.7926	1.7277	5.6279

The lower test losses do not imply that evaluation improved the weights: the splits contain different pieces and may differ in difficulty. Among these selected configurations, medium and large test perplexities are approximately 2.5% and 3.5% above small's. Appendix Figures A5–A6 present aggregate scores and per-piece distributions. These are descriptive single-seed findings, not proof of statistical reliability.

Event-family results in Appendix Figure A7 and Table A1 expose differences hidden by a single average. Small is strongest on note-on prediction; larger models do not improve every category, although some timing and pedal losses are lower. Time shifts comprise 46.13% of test targets, each note/velocity family 17.11%, and EOS only 0.0041%. The very high EOS losses therefore contribute little to the aggregate while flagging weak termination prediction. These measurements establish next-event predictive performance, not convincing long-form musical generation.

10 Conclusions, limitations, and further iterations

Evidence-based conclusions. Small gives the best observed validation performance within the two-hour budget, and the lowest test loss among the validation-selected configurations (small dropout 0.1; medium/large dropout 0.2). More parameters alone do not deliver a better predictor under this resource allocation. Medium and large fit training data strongly but overfit; dropout 0.2 lowers their validation minima and reduces later deterioration without reversing the size ranking. Small's negligible change shows that regularization strength should not be assumed architecture-independent. Efficient batching also matters: its measured 18–42% throughput improvements increase the training opportunity without increasing the allotted time.

Limits of inference. One seed and two dropout values cannot establish a global optimum or statistical reliability. The final comparison varies both size and selected dropout, and the small entry was revised after the initial test evaluation. Equal time does not equalize tokens, updates, or FLOPs; learning rate was shared rather than tuned per size. Checkpoint selection uses a small, reused validation sample and ten-minute resolution. Final scoring resets context, whereas full sliding-context likelihood would be different. The music tokenizer is lossy, restricted to piano events, and has not been validated through a listening study. No audio-quality or long-range-composition claim follows from token perplexity.

A further implementation-specific limitation is boundary exposure. With fixed full windows kept wholly inside pieces, EOS can appear only as the last target of the final admissible window, unlike interior tokens that occur at many target positions. This provides a plausible explanation for poor termination prediction,

but it has not been tested causally. Random-length training windows or deliberate tail sampling, with explicit weighting, could investigate it without assuming that a tiny aggregate loss contribution makes endings unimportant.

Next priorities. Replicate promising conditions with additional seeds and broader validation scoring; tune learning rate and regularization with comparable search budgets; test training-only pitch transposition; and compare equal-token as well as equal-time regimes. Longer context or relative position representations are motivated by musical structure [8], but add compute costs. Generate matched-prompt MIDI continuations and audit note pairing, duration, repetition, and EOS behavior. Since test results have now been inspected, further model selection requires validation and a fresh final holdout for genuinely new confirmatory claims.

Could sparse experts improve performance within the same time?

A mixture-of-experts (MoE) model replaces selected dense MLPs with a router and several experts, activating only one or a few per token. This can increase total capacity without activating all weights. Switch Transformers demonstrates substantial efficiency gains in its evaluated regimes [9]; ST-MoE studies routing/training stability [10]. Neither establishes that MoE will *almost always* outperform dense models under any rental-time cap, especially this single-A10, limited-data setting.

Routing, token dispatch, small expert workloads, imbalance, and auxiliary balancing losses can reduce realized throughput. All expert parameters and optimizer states still require storage; multi-GPU variants add communication. Sparse activation does not remove overfitting, and greater capacity may repeat the difficulty observed here. A sensible future test is a medium backbone with four top-1 experts in selected MLP layers, compared with a retuned dense baseline at the same measured A10 time. Report active and total parameters, token throughput, memory, and validation-selected loss. MoE is therefore a testable *capacity–compute tradeoff*, not a demonstrated conclusion or guaranteed route to better music.

References

[1] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel, and Douglas Eck. Enabling factorized piano music modeling and generation with the MAESTRO dataset. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1810.12247.

[2] Magenta. The MAESTRO dataset, version 3.0.0. URL https://magenta.tensorflow.org/datasets/maestro. Dataset documentation, accessed 8 November 2026.

[3] NVIDIA. NVIDIA A10 tensor core GPU. URL https://www.nvidia.com/en-us/data-center/products/a10-gpu/. Product specifications, accessed 8 November 2026.

[4] Andrej Karpathy. nanoGPT. URL https://github.com/karpathy/nanoGPT. Source implementation, accessed 8 November 2026.

[5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. URL https://arxiv.org/abs/1706.03762.

[6] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1711.05101.

[7] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. URL https://jmlr.org/papers/v15/srivastava14a.html.

[8] Cheng-Zhi Anna Huang, Ashish Vaswani, Jakob Uszkoreit, Noam Shazeer, Ian Simon, Curtis Hawthorne, Andrew M. Dai, Matthew D. Hoffman, Monica Dinculescu, and Douglas Eck. Music transformer: Generating music with long-term structure. In *International Conference on Learning Representations*, 2019. URL https://arxiv.org/abs/1809.04281.

[9] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://arxiv.org/abs/2101.03961.

[10] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. ST-MoE: Designing stable and transferable sparse expert models, 2022. URL https://arxiv.org/abs/2202.08906.

A Supplementary figures and diagnostics

A.1 Baseline training trajectories

The following enlarged figures support Sections 5–6. Points are actual recorded evaluations, not additional observations along the connecting lines.

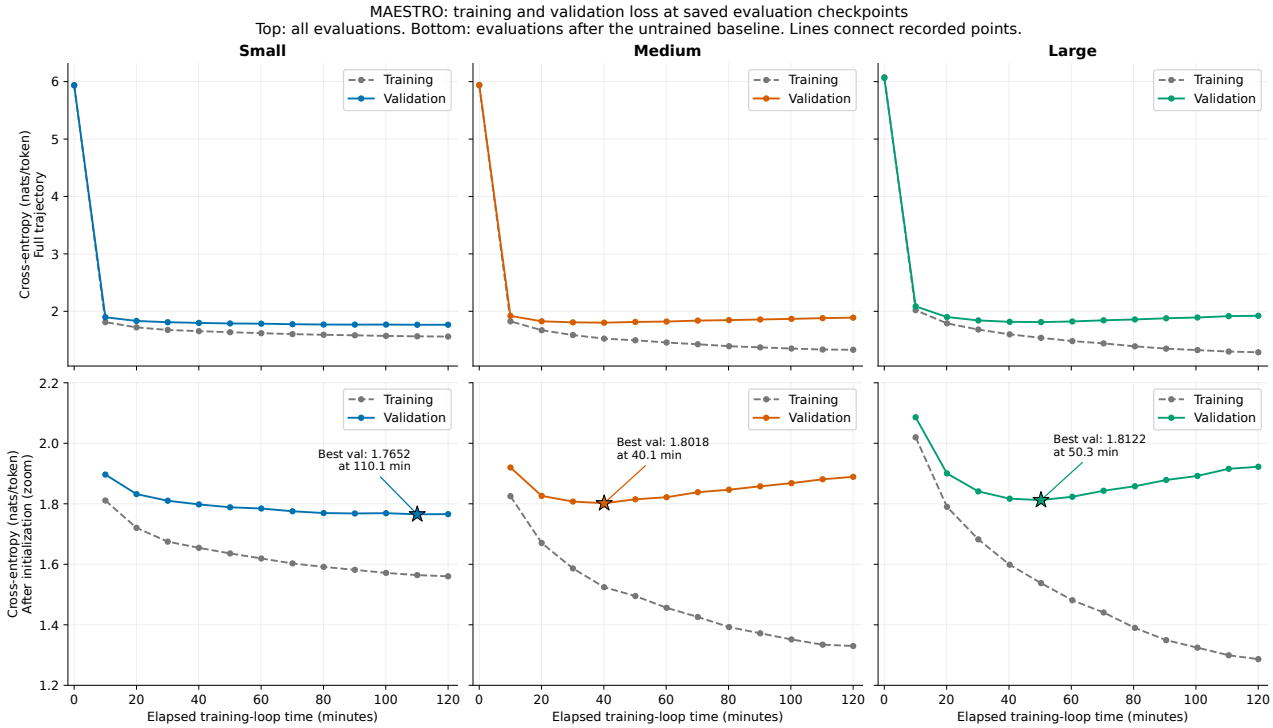


Figure A1: Baseline trajectories. Dashed grey: sampled training loss; colored solid: sampled validation loss. Stars mark the lowest recorded validation checkpoint. Connecting lines do not add measurements between checkpoints.



Figure A2: Validation comparison after initialization under equal A10 time. Small's slower, sustained improvement contrasts with the earlier minima and subsequent increases of medium and large.

A.2 Dropout intervention

These figures support Section 8. Every size receives the same nominal training-loop budget; the treatment changes dropout from 0.1 to 0.2.

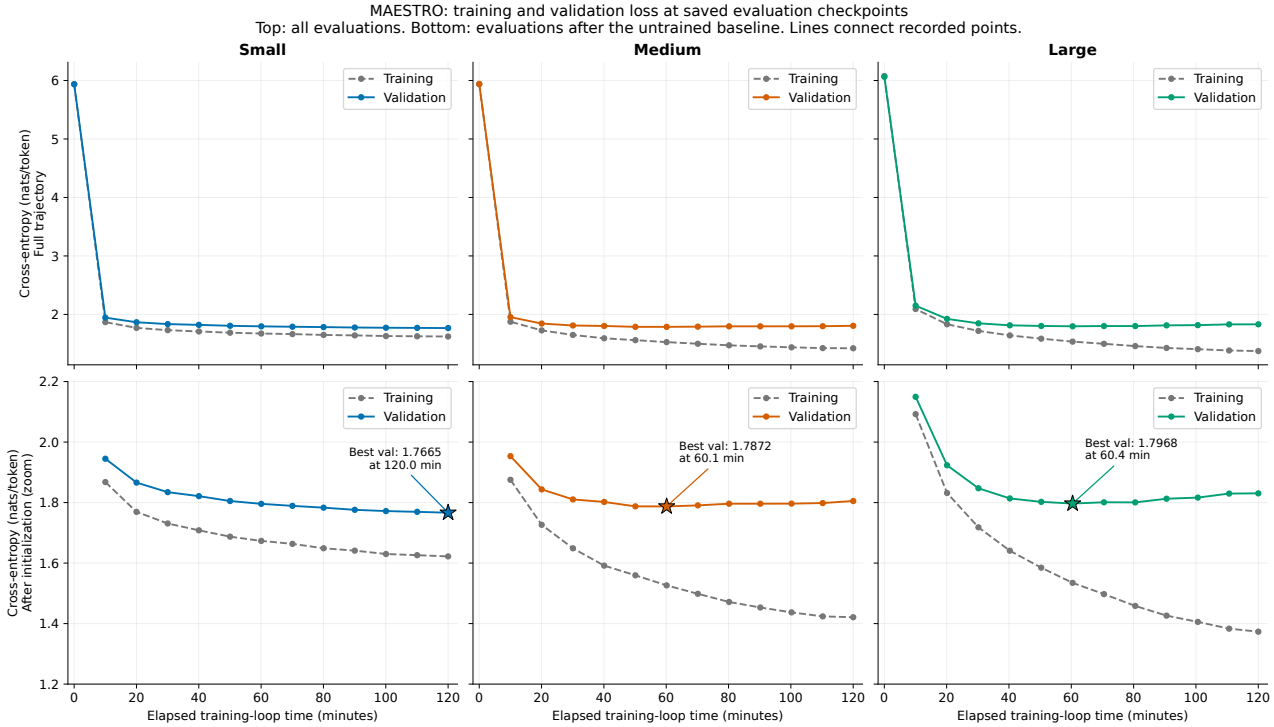


Figure A3: Full dropout-0.2 trajectories (top) and post-initialization zoom (bottom). Dashed grey is training loss; colored solid is validation loss.

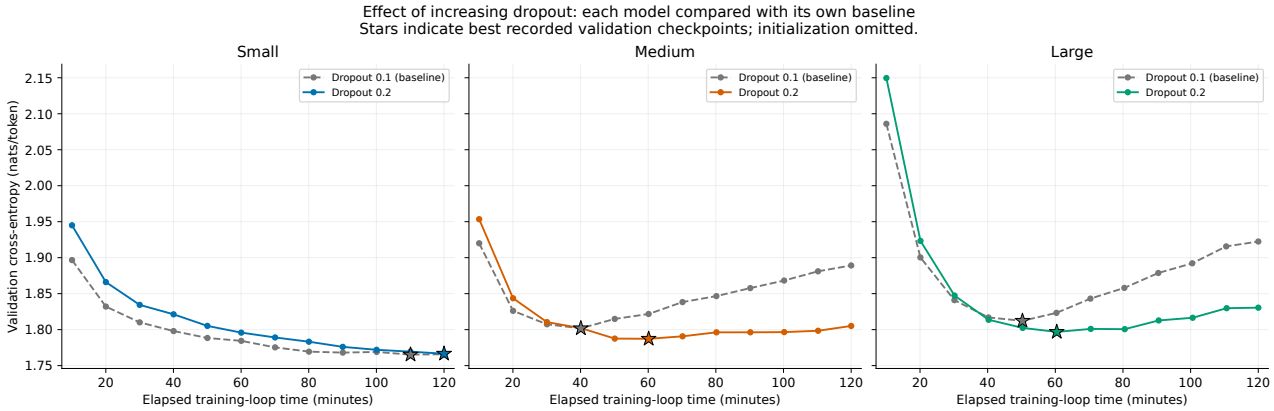


Figure A4: Validation loss with dropout 0.1 (grey dashed) and 0.2 (colored solid). Each architecture is compared with its own baseline. Stars denote recorded minima; initialization is omitted.

A.3 Final evaluation of validation-selected configurations

The figures on this page use small dropout 0.1 and medium/large dropout 0.2, all using validation-selected best checkpoints. They supersede the original all-dropout-0.2 final-comparison figures. The evaluation sequence is disclosed in Section 9.

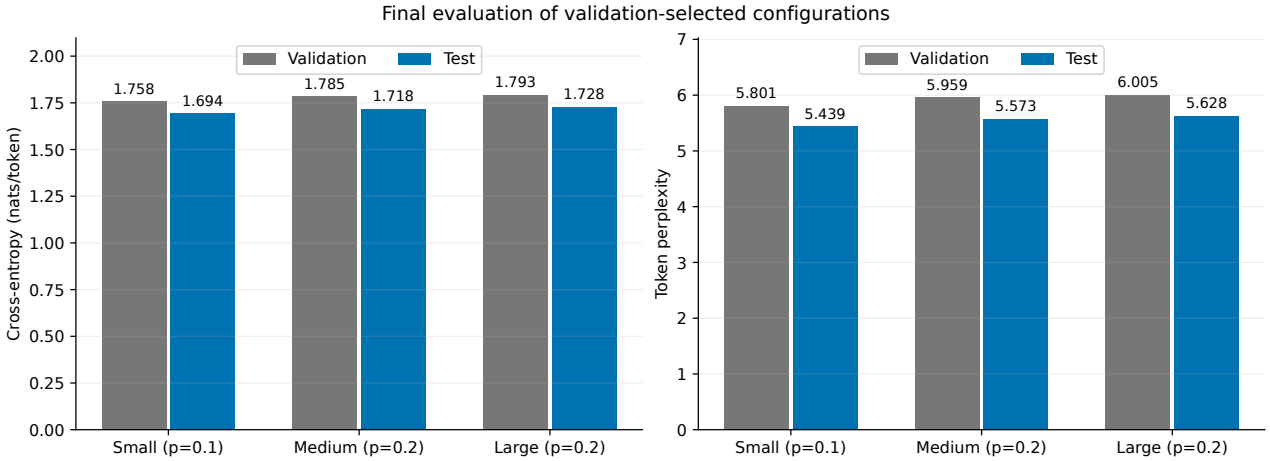


Figure A5: Full-coverage validation and test metrics for the revised primary comparison. The split scores use identical windowing but different pieces.

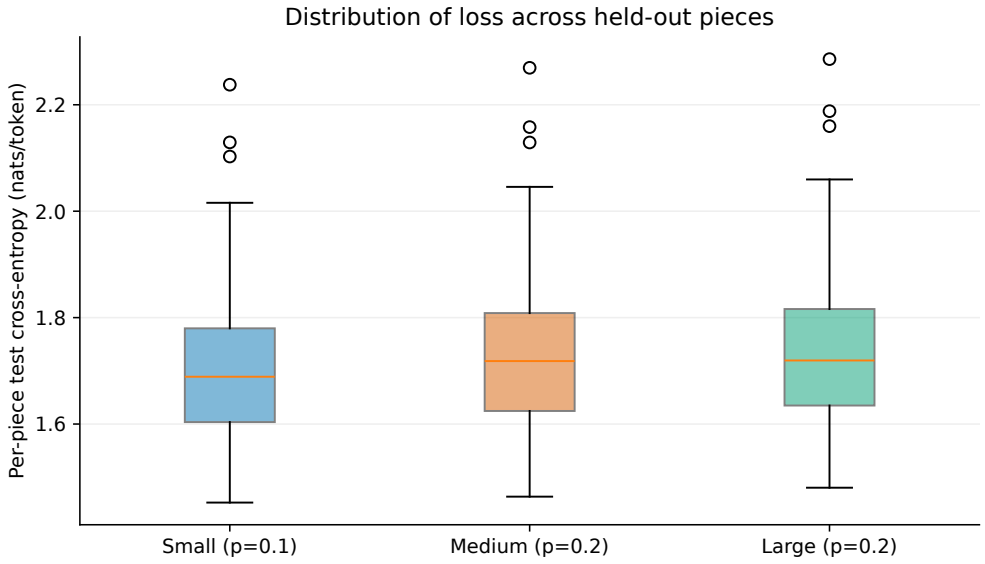


Figure A6: Per-piece test-loss distributions. The same 177 pieces appear for each model. These distributions weight pieces equally, while aggregate metrics weight tokens equally.

A.4 Event-family diagnostic losses

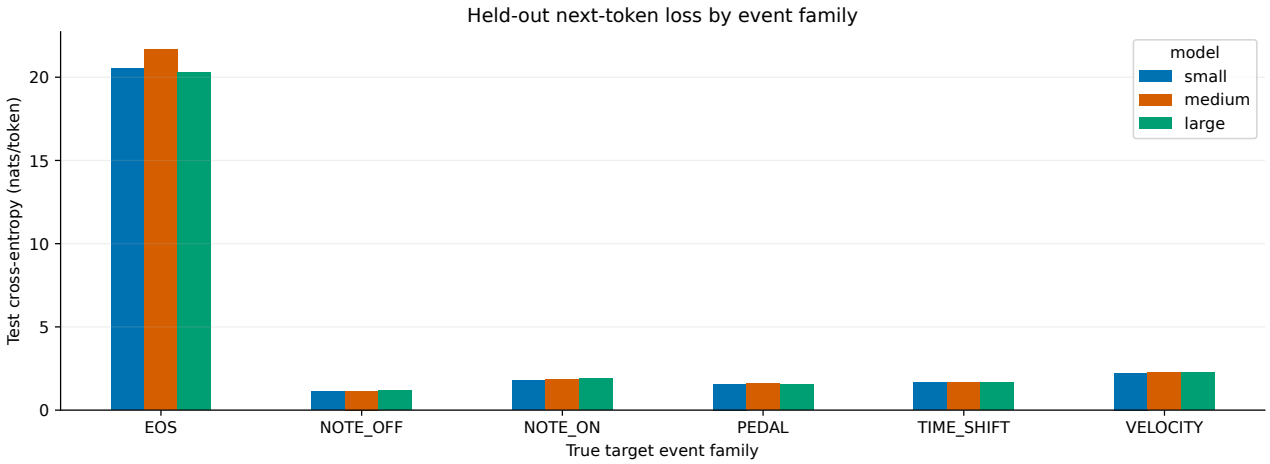


Figure A7: Mean test cross-entropy grouped by true target event family for the revised primary comparison. EOS has high loss but only 177 targets; this chart does not imply that all families contribute equally to aggregate loss.

Table A1: Per-event-family test cross-entropy for the revised primary comparison. Counts are shared across all models.

Family	Targets	Small ($p = 0.1$)	Medium ($p = 0.2$)	Large ($p = 0.2$)
NOTE_ON	741,410	1.7608	1.8615	1.8960
NOTE_OFF	741,410	1.1307	1.1544	1.1721
TIME_SHIFT	1,999,264	1.6817	1.6804	1.6805
VELOCITY	741,410	2.2357	2.2538	2.2692
PEDAL	110,421	1.5642	1.5862	1.5194
EOS	177	20.5116	21.6459	20.2792

The event counts are identical across models. Conditional category averages diagnose different prediction problems; they are not separate classification tasks. In particular, a very large EOS perplexity describes poor termination prediction under the scoring protocol, rather than the overall model perplexity. No audio or listening evaluation was performed.